

Getting our training onto Gaudi

What we did on Sol's Gaudi2 nodes and how it differs from the A100 path

Jarod DeFilippo

2026-10-07

Where the code is

- **Ark container**, <https://github.com/JarodDeFilippo/2026Fall-CSE507-Ark-Container>. My fork of the course Ark+ container, plus the DINOv3 backbone, the Demo 1 experiment arms and the Gaudi path. NOTICE lists the files I changed; the ASU license is in `LICENSE-ASU-Ark.txt`.
- **UniMiSS+**, <https://github.com/JarodDeFilippo/UniMiSS-code>. A fork of YtongXie/UniMiSS-code (MIT), plus UniMiSSPlus/Downstream/hpu_device.py, torch 2.6+ and numpy 2 fixes, a 2D multilabel VinDr-CXR mode, `tools/convert_ricord.py` and `tools/match_ricord_lists.py`, and `sol_gaudi/` (image def, wrapper, card scripts, sbatch scripts, a README with the Oct 5 results).

Drop in as they are: the three card scripts. If you rename the wrapper, fix the `./gaudi.sh` call in `gaudi_check_cards.sh`.

Copy, then edit the bind list and image path: the wrapper. In `gaudi.sh`, change the `IMAGE=` default, `--pwd /workspace/base`, the data bind (`DATA_BIND`, default `/data:/data:ro`) and `RUNTIME_ROOT` (tmp, caches, Habana logs). `unimiss_gaudi.sh` has equivalents, plus `REPO=` (one level up).

Edit: the image requirements, the sbatch scripts (build header, smoke, run/resume) and your own training loop. In a fork of the course Ark container the loop is likely done: `grep -n hpu base/accelerator.py` `experiment_scripts/common.sh` should show an HPU path and gaudi branches.

Why Gaudi

- **Access.** Lab account only (`-A grp_jliang12 -p gaudi -q public`); the class account has no Gaudi QOS.
- **Queue.** On Oct 4 a 1-card sbatch `--test-only` projected an immediate start, while our A100 jobs on the same account projected Nov 5 on public (about 10.5 hours on htc). It is not always empty; `--test-only` your real header first.
- **Long jobs.** A 7-day partition limit, against our 7.5-hour class links.
- **Hardware.** `gaudi[001-010]`, 8 Gaudi2 (HL-225) cards per node, about 18 cores per card. Ask with `-N 1 --gres=gpu:hl225:N`.
- **Speed.** One Gaudi2 in lazy mode, fp32, ViT-B/16 (DINOv3) at batch 200: 0.62 s per step against 1.76 s on one A100-80, roughly 2.8x per card (measured on gaudi003, Oct 4).

The catch is a separate software stack: Habana's own PyTorch build (2.10.0a0+gitc1e5ed4) inside its own Docker base (1.24.0, Python 3.10, numpy 2.2.6). Our A100 image (CUDA 13, PyPI torch 2.11.0 in a venv) cannot drive a Gaudi card, and pip must never replace Habana's torch, because NVIDIA packages corrupt it (per the lab's Gaudi_Demo README). That README also lists the limits: no `SyncBatchNorm`, so small per-card batches may hurt batch-norm models, and Gaudi's `torch.compile` lacks many ops. Everything here is fp32.

A100 -> Gaudi, file by file

A100 file you have	Gaudi counterpart	What changed
containers/nvidia/: apptainer.def, requirements.txt, constraints.txt nvidia.sif (downloaded)	containers/gaudi/: apptainer.def, requirements.txt, requirements_nodeps.txt build_gaudi_image.sh	Habana base image; no venv; pip constrained to the base's packages Built by you on a gaudi node, one card (22-24 min, 1.75 GB)
nvidia.sh	gaudi.sh	No --nv; maps Slurm's card index to a Habana module id; lazy mode on; binds /dev/shm and a Habana log dir; edit its paths
none	gaudi_hlsmi.sh, gaudi_check_cards.sh, gaudi_verify_busy.sh	Card-isolation checks; drop in
none	gaudi_smoke.sh	One card, 45 minutes, the whole path
run_d1_8a_class.sh, resume_d1_8a_class.sh	run_d1_8a_gaudi.sh, resume_d1_8a_gaudi.sh	Lab account, 7 days, 4 cards, -c 72, 256G, 16 workers per rank, lazy mode, card checks

Requirements: drop torch, torchvision and numpy, since the base has them. Pin the data-path libraries to the A100 versions, because unpinned pip pulled opencv 5.0.0.93 and Pillow 12.3.0 against 4.12.0.88 and 11.3.0. Starting from the demo's `gaudi-apptainer.def`, delete its `export HABANA_VISIBLE_DEVICES=all`: it selects every card, and `gaudi.sh` does not override it (it sets `HABANA_VISIBLE_MODULES`). Inside a job, `./gaudi.sh env | grep HABANA` should list your `HABANA_VISIBLE_MODULES` and no `HABANA_VISIBLE_DEVICES=all`. After the build, `/opt/pip-freeze.txt` in the image must show `torch==2.10.0a0+git...`, `numpy 2.2.6` and `habana 1.24.0.x`, or stop. Take the build header from UniMiSS's `sol_gaudi/build_image.sh`, since Ark's still says `-p htc`.

The five things that actually bit us

1. Card isolation

Slurm exports `SLURM_JOB_GPUS`, your card's index on the node, and no `HABANA_VISIBLE_*`. Habana selects cards by module id, and the index-to-module map differs per node. `torch.hpu.device_count()` says 8 even with the restriction set, and cgroups hide nothing: a 1-card job's container saw all 8 cards. So an unmodified job can silently train on someone else's card.

`gaudi.sh` maps your indices to module ids through the node's hl-smi at job start, exports `HABANA_VISIBLE_MODULES`, and refuses to run if any step fails. `gaudi_check_cards.sh` fails closed before training unless your cards are idle (at most 1536 MiB), any host-side `HABANA_VISIBLE_*` matches the allocation, and the restriction reaches the container. `gaudi_verify_busy.sh` reports 10 or 15 minutes in whether each allocated card is above 1536 MiB. Idle before plus busy after, one process per card, puts every rank on our cards.

The proof was hl-smi memory: a 1-card smoke showed its card at 42,940 MiB and the other seven at 768; a 4-card job, its four at 3,219 and the rest at 768. The threshold bit us too: at 4096 MiB the verifier, then a kill switch, cancelled a healthy job sitting at 3,219. It is 1536 now and only reports; with a small model, check that a rank clears it.

2. Lazy mode, or lose about 9 AUC points

Habana defaults to eager mode. In the lab demo, Ark+ in lazy mode matched the official result (VinDr-CXR student AUC 0.9412 against 0.9414); eager mode gave 0.8482 on the same run, with no error. `gaudi.sh` forwards `PT_HPU_LAZY_MODE` (default 1).

3. --cleanenv drops the Slurm variables

I kept `--cleanenv` from `nvidia.sh` so the container environment is identical every run. The cost: `SLURM_JOB_GPUS` never reaches the container, so the mapping and the idle check run on the host, and the wrapper forwards `PT_HPU_LAZY_MODE`, `HABANA_VISIBLE_MODULES`, `HABANA_VISIBLE_DEVICES` and `OMP_NUM_THREADS` explicitly. Anything new your code reads from the environment needs the same. And `gaudi.sh` runs unrestricted when `SLURM_JOB_GPUS` is unset, and I never checked that a `salloc` shell sets it, so look with `salloc` and train with `sbatch`.

4. Four cards are data-loader bound

At 8 workers per rank on 4 cards, MIMIC averaged 0.92 s per step with data time up to 0.8 s and roughly 29 of 72 cores busy (gaudi009, Oct 4). At 16 workers per rank it ran 0.73 s per step (1.27x) and a cycle dropped from about 49 to 40.4 minutes (gaudi005, Oct 4). Still input-bound: steps that did not wait on data took about 0.24 s.

Rule of thumb (arithmetic, not a measured optimum): 18 cores per card, with ranks times workers plus about 2 cores per rank fitting in `-c` ($4 \times 16 + 4 \times 2 = 72$). If data time still tracks batch time after that, more cards will not help. But input-bound is not CPU-bound: workers can also wait on shared storage (`/data` reads, a first pass) or another pipeline stage, and the 8-worker run left most cores idle. More workers is a cheap first experiment, not a diagnosis; before asking for more cores, check per-process CPU (`top` via `srun --overlap`, or `sstat` and `sacct`) and I/O.

5. Graphs recompile, so early speed lies

Lazy mode compiles a graph per input shape, and each pass's short last batch is a new one: about 18-20 s on 4 cards (gaudi005), a 7.44 s step on one card (gaudi003, both Oct 4). Nothing is cached across jobs, so every resume pays again, and first steps compile too. Do not read speed from early steps or a smoke: UniMiSS+'s 3D smoke ran 2.80 s per step and the real run 1.01 (gaudi008, Oct 5). And time whole passes: Ark's sampled step times missed the stalls and suggested 7-9x an A100 on 4 cards, where elapsed time gave about 2.3x on MIMIC (Oct 4).

Code changes you will actually make

Device selection with a fallback, so one codebase runs on both images. Ark has it in `base/accelerator.py` (`PROJECT_ACCELERATOR=hpu`, which `gaudi.sh` sets); UniMiSS+ needed a small new file:

```
# UniMiSSPlus/Downstream/hpu_device.py (docstring dropped)
import torch

try:
    import habana_frameworks.torch.core as htcore
    device, mark_step = torch.device('hpu'), htcore.mark_step
except ImportError:
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    mark_step = lambda: None # no-op off HPU
```

Print the device at startup and assert it in the smoke, because a failed Habana import falls back to `cpu` and trains slowly with no error.

`htcore.mark_step()`. Lazy mode accumulates ops into a graph, and `mark_step` cuts it and runs it. Call it after `loss.backward()` and after `optimizer.step()`, plus after each eval batch, after an EMA teacher update and before a checkpoint save. Ark has all of these; UniMiSS+ needed the train-step pair and the eval one.

Distributed. `hccl` backend, and `import habana_frameworks.torch.distributed.hccl` is required before `init_process_group`; do it only in the HPU branch (if `device.type == "hpu"`, like the `habana_frameworks.torch.core` import), because it fails on an image without Habana,

breaking the CUDA/CPU fallback (Ark's `base/accelerator.py` guards it). No `device_ids`, plain `torch.device("hpu")` on every rank (no `torch.cuda.set_device` counterpart), launched with `python -m torch.distributed.run --standalone --nproc-per-node N`, because `torchrun` is not implemented on Gaudi. Ark has it; UniMiSS+ runs on one card.

CUDA-only calls. `.cuda()` becomes `.to(device)` (UniMiSS+ only; Ark's accelerator layer covers it). Set `pin_memory` only on CUDA, since `True` on HPU is unverified. Save CPU copies of state dicts (a demo README rule; the files become device-neutral). `cudnn` flags are harmless.

torch.load. Load to CPU with `torch.load(path, map_location="cpu")`, because a checkpoint saved from HPU or CUDA tensors otherwise tries to deserialize onto a device that may not exist in the loading process. For a plain state dict, or one inside a wrapper dict, pass it to `model.load_state_dict(...)`, move the model with `model.to(device)` once, build the optimizer from the moved parameters, then call `optimizer.load_state_dict(...)` for a resume; never `.to()` the loaded dict. This one is universal, not Gaudi-specific: torch 2.6 made `weights_only=True` the default, and both images ship newer torch. The rule: load anything you did not produce yourself with the restricted loader (`weights_only=True`), because unpickling a file runs arbitrary code as you. If it refuses a class the checkpoint needs, allowlist that class with `torch.serialization.add_safe_globals([...])` and keep the restricted loader; the usual offenders are numpy's `scalar` and `dtype`. `weights_only=False` is only for checkpoints your own code wrote. A matching SHA-256 proves you have the file the publisher posted, not that it is safe to unpickle, so it does not change the rule. Both of my ports predate this rule: Ark+ loads every checkpoint with `weights_only=False`, including the downloaded DINOv3 weights, and the UniMiSS+ port does the same on its five loads of the authors' `UniMissPlus.pth`. Neither needs to: in a check on my Mac on Oct 6 (torch 2.12), both files loaded under the restricted loader, `UniMissPlus.pth` after allowlisting numpy's `scalar` and `dtype`. Treat those `False` flags as debt to remove in your fork, not a pattern to copy.

Also universal: `np.int()` becomes `int()` under numpy 2.

Smoke test

Run the whole path on one card first. A short version, with placeholder names (Ark's `gaudi_smoke.sh` adds host checks, a strict weight-load check and a fuller verdict):

```
#!/usr/bin/env bash
#SBATCH -A grp_jliang12
#SBATCH -p gaudi
#SBATCH -q public
#SBATCH -N 1
#SBATCH --gres=gpu:hl225:1
#SBATCH -c 18
#SBATCH --mem=96G
#SBATCH -t 00:45:00
set -uo pipefail
cd /scratch/$USER/my-fork || exit 1
[[ -f containers/gaudi/gaudi.sif && ! -e my_exp_seed_1 ]] || exit 1
hl-smi -Q index,module_id,memory.used -f csv
./gaudi_check_cards.sh || exit 1
./gaudi_verify_busy.sh 600 &
timeout --kill-after=60 1800 ./experiment_scripts/my_exp/start_my_exp.sh 1 gaudi
rc=$?
log=my_exp_seed_1/train.log
grep -qi 'device: hpu' "$log" || { echo "no hpu device line in $log"; exit 1; }
n=$(grep -c 'BT=' "$log" 2>/dev/null)
[[ ${n:-0} -gt 0 && ( $rc -eq 0 || $rc -eq 124 ) ]] && echo "SMOKE OK" || exit 1
```

The device check comes first because a CPU fallback still logs `BT=`. Exit 124 counts only with `BT=` lines, since a hung run also ends in 124.

A pass, in `slurm-<id>.out`: idle baseline OK, a line like `gaudi.sh: SLURM_JOB_GPUS=3 -> HABANA_VISIBLE_MODULES=7` (your numbers), `restriction` OK, then at 10 minutes `card use` OK with your card above 1536 MiB in the `hl-smi` table, then `SMOKE` OK. Delete the throwaway run directory afterwards, or the next start refuses it. Chain a smoke to its build with `afterok:<build>`, since Gaudi jobs can start before the image exists.

When it breaks

Symptom	Likely cause	Where to look
“Requested node configuration is not available”	No card requested	Add <code>--gres=gpu:hl225:1</code> , even for builds
Job launches 8 ranks	Rank count from <code>device_count()</code>	Count <code>SLURM_JOB_GPUS</code> entries
no <code>HABANA_VISIBLE_*</code> reached the container	No <code>SLURM_JOB_GPUS</code> , or not forwarded	The wrapper’s mapping line
allocated card index N already shows ... MiB in use	Leftover process	<code>hl-smi</code> ; kill yours by PID
<code>synStatus=8</code> [Device not found]	An interrupted Gaudi process is still alive	Find its PID with <code>hl-smi</code> , kill it
<code>WARNING: card use check</code>	Threshold too high for your model, or a rank on the wrong card	The <code>hl-smi</code> table above it; <code>scancel</code> by hand if confirmed
Metrics about 9 points low, no error	Eager mode	<code>PT_HPU_*</code> lines in the container probe
Device line says <code>cpu</code> or <code>cuda</code>	Habana import failed	Run the import in the image by hand
Freeze shows another torch or numpy	A requirement wants other base packages	Constraints, <code>--no-deps</code> , or a compatible pin

Results so far

Demo 1, arm B: Ark+ with a DINOv3 ViT-B/16 backbone and AdamW, cyclic over four datasets, on 4 Gaudi2 cards (gaudi005, evaluations read Oct 5-6). It runs about 41.7 minutes per cycle and should finish 200 cycles around Oct 10, inside its 7-day job. The teacher’s mean AUC peaked at cycles 10-20 and has drifted down since, while training loss keeps falling.

Teacher mAUC	Peak (cycle)	c55	c60	c60 vs peak
VinDr-CXR	0.959 (c10)	0.944	0.944	-1.5 pts
ChestX-ray14	0.832 (c15)	0.762	0.756	-7.6 pts
CheXpert	0.902 (c10)	0.796	0.794	-10.8 pts
MIMIC	0.796 (c15)	0.720	0.717	-7.9 pts

c10 values are after the ChestX-ray14 pass; one seed, center crop.

Practice 3, UniMiSS+, one Gaudi2 card (gaudi008 and gaudi009, Oct 5). 2D VinDr-CXR, six labels, 30 epochs in about 46 minutes: test mean AUC 0.848. 3D RICORD early-stopped after epoch 69 (patience 30), with best validation AUC 0.886 at epoch 39. Test AUC is 0.909 from `Final.pth`, which holds the epoch-68 weights; `Best.pth` (epoch 39) is untested.

More detail

The long guide walks the port step by step, with every number’s provenance (job, node, date), the full debugging table and the sources: <https://github.com/JarodDeFilippo/2026Fall-CSE507-Ark-Container/blob/>

main/docs/sol-gaudi-guide.pdf. This guide is docs/audi-vs-a100.pdf there too. Questions to me.